Table des matières

Cours 09 — Ingérer un flux d'événements 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Régler le seuil de l'alerte · Sources 2
Étape 2 — Le journal du direct · Ingestion 3
Étape 3 — Écrire `consommer.py` · Ingestion 4
Étape 4 — Consommer la matinée · Ingestion 6
Étape 5 — Le reste de la journée, repris au décalage · Ingestion 7
Étape 6 — Une panne avant le décalage · Ingestion 7
Étape 7 — Le direct contre le fichier du jour · Ingestion 8
Étape 8 — Versionner · Ingestion 9
À retenir 9

Cours 09 — Ingérer un flux d'événements

Cours 9/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
	✓ ici	

Ce que vous allez faire

Jusqu'ici, la courbe d'une journée arrive en un seul fichier, et le pipeline la charge la nuit : c'est le cold path. Le logiciel du torréfacteur peut aussi publier chaque lecture au moment où ses sondes la prennent : c'est le hot path, celui qui sait pendant que le lot torréfie encore. Boréal le monte au même cours avec un courtier MQTT et Kafka ; La Brûlerie en fait une petite version, sans courtier, avec un journal en fichier où chaque message s'ajoute au bout. Vous réglez d'abord le seuil d'une alerte sur le mois déjà chargé, puis vous écrivez `consommer.py`, qui lit le journal là où il s'était arrêté, écarte les messages reçus deux fois, et lève l'alerte à l'heure où la chose s'est produite.

© 2026 iAlexMG. Tous droits réservés.

le seuil, mesuré sur le mois → le journal du 10 juin, jusqu'à 8 h 30 → `consommer.py`, depuis le début du journal → le reste de la journée, repris au décalage → une panne qui fait tout relire, sans rien doubler → le direct contre le fichier du jour

Le code complet est dans `Codes_09_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, Python et Git ; la seconde pour le notebook, qui s'ouvre à l'étape 1. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir /b
REM Attendu : 01_de, plateforme et sources, plus ce qu'Analytics Engineering y a laissé.
```

Ce cours part de l'entrepôt chargé jusqu'au 30 juin, tel que le cours 08 le laisse. S'il manque, reprenez la fin du cours 08.

Étape 1 — Régler le seuil de l'alerte · Sources

Une alerte se règle sur ce que la source fait d'habitude. Trop basse, elle sonne pour un lot sur deux, et une alerte qui sonne tout le temps ne se lit plus ; trop haute, elle ne sonne jamais. On la règle donc sur le mois que le cold path a déjà chargé. Celle de ce cours surveille le grain qui chauffe trop : le profil le plus foncé que le café torréfie en juin, le moyen-foncé, vise une sortie à 217 °C (le dictionnaire des sources, section 4, « Les profils de torréfaction »).

1.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 09 ». Écrire les deux premières cellules, comme au cours 03 : la première attache l'entrepôt en lecture seule, l'exécuter ; la seconde le détache, l'écrire sans l'exécuter.

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS entrepot (READ_ONLY);
-- Attendu : entrepot paraît sous Attached databases.
```

```
DETACH DATABASE IF EXISTS entrepot;
```

1.2 Compter, pour chaque seuil de 215 à 220 °C, les lots du mois dont le grain l'a dépassé, dans une cellule neuve. `range(215, 221)` rend les entiers de 215 à 220, la borne de fin exclue, et `CROSS JOIN` les colle à chaque lot, comme au cours 07.

```
-- 1. la plus haute température du grain de chaque lot
WITH lots AS (
    SELECT lot_id, MAX(CAST(temp_grain AS DECIMAL(5, 1))) AS grain_max
```


© 2026 iAlexMG. Tous droits réservés.

```
    FROM entrepot.raw_courbes
    GROUP BY lot_id
)
-- 2. pour chaque seuil, les lots qui l'ont dépassé
SELECT seuil, COUNT(*) FILTER (WHERE grain_max > ____) AS lots
FROM lots CROSS JOIN range(215, 221) AS r(seuil)
GROUP BY seuil
ORDER BY seuil;
-- Attendu : 6 lignes : 215 à 18 lots, 216 à 15, 217 à 11, 218 à 5, 219 à 1, 220 à 0.
```

À 217 °C, la sortie visée elle-même, l'alerte sonnerait pour 11 lots, presque deux sur trois des 18 lots moyen-foncé : elle ne dirait plus rien. À 219 °C, pour un seul. À 218 °C, un degré au-dessus de la sortie visée, elle sonne pour 5 lots sur 47 : c'est le seuil de ce cours.

1.3 Voir ces cinq lots, et leur jour.

```
SELECT l.lot_id, CAST(l.debut AS DATE) AS jour, MAX(CAST(c.temp_grain AS DECIMAL(5, 1))) AS grain_max
FROM entrepot.raw_courbes AS c
JOIN entrepot.raw_lots AS l USING (lot_id)
GROUP BY l.lot_id, jour
HAVING grain_max > 218
ORDER BY l.lot_id;
-- Attendu : 5 lignes : L-2025-0220 le 2025-06-03 à 218.3, L-2025-0229 le 2025-06-10 à 218.4,
-- L-2025-0242 le 2025-06-17 à 218.6, L-2025-0247 le 2025-06-20 à 218.8, L-2025-0259 le 2025-06-27 à 219.1.
```

Une alerte par semaine de torréfaction, à peu près. Le 10 juin, la plus grosse journée du mois avec ses neuf lots, en compte une : c'est la journée que le direct va rejouer.

Étape 2 — Le journal du direct · Ingestion

Le hot path ne lit plus des fichiers, mais des messages : un par lecture des sondes, dans l'ordre où elles se font. Entre le logiciel qui les publie et le programme qui les lit, un journal les garde. Chaque message s'y ajoute au bout, rien ne s'y efface, et chaque lecteur y tient sa place : son décalage, le rang du premier message qu'il n'a pas encore lu. C'est ce que fait Kafka chez Boréal. Ici, le journal est un fichier, `plateforme\direct\journal.jsonl`, qui porte une ligne JSON par message.

Le rejoueur fourni, `rejouer_direct.py`, joue le logiciel du torréfacteur : il écrit une journée de courbes, message par message, chacun daté de l'heure de l'événement, le début du lot plus sa seconde. Comme un vrai éditeur, il renvoie un message quand il ne sait pas s'il est arrivé : un message sur vingt part deux fois, et, quand il reprend, il renvoie d'abord son dernier message. C'est la livraison « au moins une fois » : rien ne manque, mais un message peut arriver en double. Ses options, avec leur valeur de départ, sont dans `--help` ; les trois qui servent ici sont le jour rejoué (le 10 juin), l'heure où il s'arrête (la fin de la journée) et la fréquence des renvois (un message sur vingt).

2.1 Rejouer le matin du 10 juin, jusqu'à 8 h 30, dans la première fenêtre.

© 2026 iAlexMG. Tous droits réservés.

```
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\rejouer_direct.py --jusqua 08:30
REM Attendu : Rejoué : 2025-06-10, de 07:00:00 à 08:30:00, puis Messages écrits : 447, dont 21 renvois,
REM puis Le journal compte 447 messages.
```

Si la commande s'arrête sur	La cause, et le geste
`can't open file '…rejouer_direct.py'`	la fenêtre n'est pas dans `Projet_Brulerie`, ou `Projet_Brulerie` n'est pas dans `Data_Domain` : `cd` vers `Projet_Brulerie`, puis la commande
`Aucune courbe pour 2025-06-11 : le torréfacteur n'a pas tourné ce jour-là`	un `--jour` sans torréfaction ; le message liste les huit jours où il a tourné

2.2 Lire le début du journal, dans le notebook. `read_json` lit un fichier de lignes JSON comme une table, une colonne par champ.

```
SELECT * FROM read_json('plateforme/direct/journal.jsonl') LIMIT 3;
-- Attendu : 3 lignes du lot L-2025-0223, aux secondes 0, 10 et 20, de 2025-06-10 07:00:00 à 07:00:20 ;
-- la première porte CHARGE.
```

2.3 Compter les messages, et les lectures qui y paraissent plus d'une fois.

```
-- 1. les messages du journal, puis les lectures (lot, seconde) qui y paraissent plus d'une fois
SELECT COUNT(*) AS messages,
       (SELECT COUNT(*) FROM (SELECT lot_id, seconde FROM read_json('plateforme/direct/journal.jsonl')
                              GROUP BY lot_id, seconde HAVING COUNT(*) > 1)) AS en_double
FROM read_json('plateforme/direct/journal.jsonl');
-- Attendu : 447 messages, 21 en double.
```

Un message n'a pas d'identifiant : ce qui le désigne, c'est le fait qu'il porte, une lecture d'un lot à une seconde. C'est cette clé, (lot, seconde), qui va écarter les doublons.

Étape 3 — Écrire `consommer.py` · Ingestion

Le consommateur lit ce qui est arrivé, puis s'arrête. Relancé toutes les quelques secondes, il ferait le direct par petits lots : c'est le déclencheur `AvailableNow` de Databricks. Un consommateur qui ne s'arrête jamais, abonné à Kafka, reste à Boréal. Quatre décisions font le script :

- le décalage : le consommateur garde dans `plateforme\direct\decalage.txt` le rang du premier message qu'il n'a pas lu ; relancé, il ne lit que la suite ;
- les doublons : la table du direct, `direct_courbes`, a pour clé primaire (lot, seconde), un fait par ligne. `INSERT … ON CONFLICT DO NOTHING` n'écrit pas une ligne dont la clé existe déjà, et rend le nombre de lignes écrites, 1 ou 0. La clé primaire s'apprend au Cours 03 de la formation SQL avancé, `Theorie_03_SQL.md`, section 3, et `ON CONFLICT`, à sa section 6 ;

© 2026 iAlexMG. Tous droits réservés.

- l'alerte : jugée sur les seules lectures nouvelles, et datée de l'heure de l'événement, que le message porte. Le rejoueur écrit une matinée en une fraction de seconde : l'heure où le programme lit un message ne dit rien de ce qui s'est passé au torréfacteur ;
- l'ordre : le décalage s'enregistre après l'écriture. Une panne entre les deux fait relire des messages déjà écrits, que la clé écarte ; l'ordre inverse perdrait ceux que la panne a coupés.

Pour remplir	**L'outil**
la clé d'une lecture	`PRIMARY KEY (lot_id, seconde)`
la suite du journal	`lignes[decalage:]`, les lignes à partir du rang `decalage`, le premier rang valant 0
écrire sans doubler	`ON CONFLICT DO NOTHING` ; `.fetchone()[0]` rend les lignes écrites
passer un message à la requête	`$lot_id`, `$seconde`… : une requête à paramètres nommés prend ses valeurs dans un dictionnaire, chacune sous son nom, et le message lu par `json.loads` en est un
l'heure de l'événement	le champ `instant` du message
enregistrer le décalage	`DECALAGE.write_text(str(…))`, le rang qui suit le dernier message lu

3.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\consommer.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/consommer.py : lire le journal du direct depuis son décalage, sans doublon, et alerter
import json
import sys
from pathlib import Path
import duckdb

JOURNAL, DECALAGE = Path("plateforme/direct/journal.jsonl"), Path("plateforme/direct/decalage.txt")

def consommer(con, seuil):
    con.execute("CREATE TABLE IF NOT EXISTS direct_courbes (lot_id TEXT, seconde INTEGER, instant TIMESTAMP, "
                "temp_grain DECIMAL(5, 1), temp_air DECIMAL(5, 1), evenement TEXT, PRIMARY KEY (____))")
    # 1. reprendre au décalage : le rang du premier message pas encore lu
    decalage = int(DECALAGE.read_text()) if DECALAGE.exists() else 0
    messages = [json.loads(l) for l in JOURNAL.read_text(encoding="utf-8").splitlines()[____]]
```


© 2026 iAlexMG. Tous droits réservés.

```
        nouveaux = 0
        for m in messages:
            # 2. une lecture déjà écrite : la clé (lot, seconde) la refuse, et l'écriture rend 0
            ecrit = con.execute("INSERT INTO direct_courbes VALUES ($lot_id, $seconde, $instant, $temp_grain, "
                                "$temp_air, $evenement) ON CONFLICT ____", m).fetchone()[0]
            nouveaux += ecrit
            # 3. l'alerte, datée de l'heure de l'événement, que le message porte
            if ecrit and m["temp_grain"] > seuil:
                print(f"ALERTE {m['____']} : lot {m['lot_id']}, grain à {m['temp_grain']} °C")
        # 4. le décalage, enregistré après l'écriture : une panne entre les deux fait relire, sans doubler
        print(f"{len(messages)} messages lus depuis le décalage {decalage}, {nouveaux} lectures nouvelles")

    if __name__ == "__main__":
        with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
            consommer(con, float(sys.argv[1]))
```

Enregistrer, puis fermer le Bloc-notes. Le seuil se donne à la commande : le script ne le fige pas.

Étape 4 — Consommer la matinée · Ingestion

Le consommateur écrit dans l'entrepôt : chaque passage se fait en trois temps, comme un chargement.

4.1 Dans le notebook, relancer la seconde cellule, le `DETACH`. Puis, dans la première fenêtre :

```
python 01_de\consommer.py 218
REM Attendu : 447 messages lus depuis le décalage 0, 426 lectures nouvelles ; aucune alerte.
type plateforme\direct\decalage.txt
REM Attendu : 447.
```

Si la commande s'arrête sur	La cause, et le geste
`IO Error: Cannot open file "…\brulerie.duckdb"`, puis `File is already open in …\duckdb.exe`	le notebook tient l'entrepôt : relancer le `DETACH`, puis la commande
`IndexError: list index out of range`	le seuil manque à la commande : `python 01_de\consommer.py 218`
`Invalid Input Error: Values were not provided for the following prepared statement parameters: …`	un paramètre nommé ne porte pas le nom d'un champ du message ; les six sont `lot_id`, `seconde`, `instant`, `temp_grain`, `temp_air` et `evenement`
`Binder Error: There are no UNIQUE/PRIMARY KEY constraints that refer to this table`	la table a été créée sans sa clé, et `CREATE TABLE IF NOT EXISTS` ne la refait pas : corriger le script, puis `duckdb`

© 2026 iAlexMG. Tous droits réservés.

Si la commande s'arrête sur	La cause, et le geste
	`plateforme\entrepot\brulerie.duckdb -c "DROP TABLE direct_courbes"` et `del plateforme\direct\decalage.txt`, et relancer

4.2 Relancer la première cellule du notebook, l'`ATTACH`, puis compter ce que le direct a écrit.

```
SELECT COUNT(*) AS lectures, MIN(instant) AS premiere, MAX(instant) AS derniere
FROM entrepot.direct_courbes;
-- Attendu : 426 lectures, de 2025-06-10 07:00:00 à 2025-06-10 08:30:00.
```

447 messages, 426 lectures : les 21 renvois se sont arrêtés à la clé, sans une ligne de code qui les cherche.

Étape 5 — Le reste de la journée, repris au décalage · Ingestion

5.1 Rejouer la fin du 10 juin, dans la première fenêtre. Sans `--jusqua`, le rejoueur va jusqu'au bout de la journée.

```
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\rejouer_direct.py
REM Attendu : Reprise après 2025-06-10 08:30:00 : ce dernier message est renvoyé, faute d'accusé.
REM Puis Rejoué : de 08:30:10 à 09:31:20, Messages écrits : 315, dont 16 renvois,
REM et Le journal compte 762 messages.
```

5.2 Dans le notebook, le `DETACH` ; puis, dans la première fenêtre, consommer la suite, deux fois.

```
python 01_de\consommer.py 218
REM Attendu : ALERTE 2025-06-10 08:56:00 : lot L-2025-0229, grain à 218.4 °C, puis
REM 315 messages lus depuis le décalage 447, 299 lectures nouvelles.
python 01_de\consommer.py 218
REM Attendu : 0 messages lus depuis le décalage 762, 0 lectures nouvelles.
```

Le consommateur n'a lu que la suite, les 315 messages après son décalage. Le premier, le renvoi du message de 8 h 30, s'est arrêté à la clé avec les 15 autres renvois. Et l'alerte porte 8 h 56, l'heure où le lot L-2025-0229 est sorti du tambour à 218,4 °C, et non l'heure de votre poste : le torréfacteur le sait pendant que son lot refroidit, alors que le cold path ne l'aurait vu qu'au chargement de nuit de la courbe du 10 juin.

Étape 6 — Une panne avant le décalage · Ingestion

Le pire moment d'une panne, pour un consommateur, tombe entre l'écriture des lectures et celle du décalage : au redémarrage, il reprend à un décalage trop ancien et relit ce qu'il avait déjà écrit. Effacer le décalage fait la même chose, en plus grand : tout le journal est relu.

6.1 Effacer le décalage, puis consommer, dans la première fenêtre. Le notebook est détaché depuis l'étape 5.

© 2026 iAlexMG. Tous droits réservés.

```
del plateforme\direct\decalage.txt
python 01_de\consommer.py 218
REM Attendu : 762 messages lus depuis le décalage 0, 0 lectures nouvelles ; aucune alerte.
```

Relu en entier, le journal n'a rien doublé, et l'alerte ne s'est pas levée une seconde fois.

Garantie	Ce qu'elle promet	Comment on l'obtient
au plus une fois	rien n'est doublé, mais un message peut manquer	le décalage enregistré avant l'écriture : jamais employé ici
au moins une fois	rien ne manque, mais un message peut arriver en double	le renvoi de l'éditeur, et le décalage enregistré après l'écriture
exactement une fois, en effet	chaque fait se retrouve une seule fois dans la table	au moins une fois, plus une écriture qui ne change rien la seconde fois, la clé et `ON CONFLICT DO NOTHING`

Le décalage et la table vont ensemble : un entrepôt refait de zéro demande d'effacer aussi `plateforme\direct\decalage.txt`, et le consommateur relit tout le journal, sans rien perdre.

Étape 7 — Le direct contre le fichier du jour · Ingestion

Le 10 juin est arrivé deux fois : en direct, message par message, et le soir, en un fichier, que le pipeline a chargé dans `raw_courbes`. Les deux chemins doivent dire la même chose. Boréal appelle Lambda cette architecture à deux chemins : le hot path répond tout de suite, le cold path recalcule la vérité, et les deux doivent tomber d'accord.

7.1 Relancer l'`ATTACH`, puis comparer, dans une cellule neuve.

```
-- 1. les lectures du 10 juin, telles que le fichier du jour les a livrées au cold path
WITH fichier AS (
    SELECT lot_id, CAST(seconde AS INTEGER) AS seconde, CAST(temp_grain AS DECIMAL(5, 1)) AS temp_grain,
           CAST(temp_air AS DECIMAL(5, 1)) AS temp_air, evenement
    FROM entrepot.raw_courbes
    WHERE CONTAINS(filename, '2025-06-10')
),
direct AS (SELECT lot_id, seconde, temp_grain, temp_air, evenement FROM entrepot.direct_courbes)
-- 2. dans les deux sens : ce qui manque au direct, puis ce qu'il a en trop
SELECT (SELECT COUNT(*) FROM direct) AS lectures,
       (SELECT COUNT(*) FROM (SELECT * FROM fichier EXCEPT ALL SELECT * FROM direct)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM direct ____ SELECT * FROM fichier)) AS en_trop;
-- Attendu : 725 lectures, 0 et 0.
```


© 2026 iAlexMG. Tous droits réservés.

7.2 Lire la lecture qui a levé l'alerte.

```
SELECT * FROM entrepot.direct_courbes WHERE temp_grain > 218;
-- Attendu : 1 ligne : L-2025-0229, seconde 920, 2025-06-10 08:56:00, 218.4, 247.2, SORTIE.
```

725 lectures de part et d'autre, celles des neuf lots du 10 juin, et rien de plus : le direct a tout reçu, une seule fois. L'alerte tombe à la sortie du lot, au moment où le grain est le plus chaud.

7.3 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 8 — Versionner · Ingestion

Le journal et son décalage vivent dans `plateforme\direct\`, hors du dépôt : ce sont des données, pas du code.

8.1 Valider le script, dans la première fenêtre.

```
git -C 01_de add consommer.py
git -C 01_de commit -m "Cours 09 : consommer le journal du direct, sans doublon, avec son alerte"
REM Attendu : 1 file changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Un journal garde chaque message au bout, et le lecteur y tient sa place, le décalage : relancé, il ne lit que la suite.
- Chaque maillon livre au moins une fois ; la clé de la table et une écriture qui ne double rien font que chaque lecture n'y entre qu'une fois, même relue.
- Une alerte se règle sur ce que la source fait d'habitude, et se date de l'heure de l'événement, que le message porte, jamais de l'heure où on le lit.

© 2026 iAlexMG. Tous droits réservés.